CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

EXPERIMENT – 9

Name: Hari krishan	**UID:** 24BAI70898
Section/Group: 24AIT_NTPP 3 (A)	**Branch/Semester:** CSE_AIML (CS221) / 5th
Subject Code: 24 CSP-337	**Subject Name:** Full Stack - II

Experiment – 9.1

1) AIM: To design and implement a Continuous Integration pipeline that automates building and testing of applications.

2) OBJECTIVE

1. To understand continuous Integration concepts.
2. To implement unit and integration testing.
3. To perform contrast testing between services.
4. To automate testing workflows.

3) SOFTWARE REQUIREMENT

1. Java(JDK 17+)
2. Spring Boot
3. IDE(VSCode/Eclipse)
4. Junit 5
5. MockMVC
6. Pack (for contract testing)
7. Test containers
8. GitHub account

4) THEORY

Continuous Integration (CI) is a development practice where code changes are automatically built and tested whenever they are pushed to a repository. This helps in detecting bugs early and maintaining code quality.

In this experiment, different levels of testing are implemented:

- **Unit Testing:** validates individual methods
- **Integration Testing** ensures APIs function correctly
- **Contract Testing** verifies communication between frontend and backend

Additionally, Testcontainers are used to simulate real infrastructure like databases or Kafka during testing, making tests more reliable and closer to production environments.

Automating these tests ensures that every code change is validated before integration, improving system stability and reducing manual effort.

5) IMPLEMENTATION/PROCEDURE

1. Start by setting up a Docker and configuring Kafka.
2. Build the spring boot application using maven, dependencies and suitable java version
3. Create a database in MySQL to handle data.
4. Write tests using Junit 5
5. Implement API tests using MockMvc
6. Define contract tests using pact

7. Integrate Test Containers for databases/kafka
8. Configure CI pipeline to run tests automatically

6) CODE

Configured the datasource and JPA settings in `application.properties`

```
spring.application.name=ci-pipeline

spring.datasource.url=jdbc:mysql://localhost:3306/cipipeline?useSSL=false&allowPublicKeyRetrieval=true&server
Timezone=UTC
spring.datasource.username=${DB_USERNAME:root}
spring.datasource.password=${DB_PASSWORD:password}
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

spring.jpa.hibernate.ddl-auto=update
spring.jpa.database-platform=org.hibernate.dialect.MySQLDialect spring.jpa.show-
sql=true
```

unit Testing(Added service tests for saving a product and looking it up when it exists or is missing in `ProductServiceTest.java`.)

```
package com.example.ci_pipeline; import static
org.junit.jupiter.api.Assertions.assertEquals; import static
org.junit.jupiter.api.Assertions.assertSame; import static
org.junit.jupiter.api.Assertions.assertTrue; import static
org.mockito.Mockito.verify; import static
org.mockito.Mockito.when; import java.util.Optional;

import org.junit.jupiter.api.Test; import
org.junit.jupiter.api.extension.ExtendWith; import
org.mockito.InjectMocks; import
org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

import com.example.ci_pipeline.product.Product; import
com.example.ci_pipeline.repository.ProductRepository;
import com.example.ci_pipeline.service.ProductService;

@ExtendWith(MockitoExtension.class)
public class ProductServiceTest {

    @Mock
    private ProductRepository productRepository;

    @InjectMocks
    private ProductService productService;

    @Test
```

```
    void createSavesAndReturnsProduct() {
      Product product = new Product("Keyboard", 49.99);
      Product savedProduct = new Product("Keyboard", 49.99);
      savedProduct.setId(1L);
      when(productRepository.save(product)).thenReturn(savedProduct);

      Product result = productService.create(product);

      assertSame(savedProduct, result);
    verify(productRepository).save(product); }

    @Test
    void findByIdReturnsProductWhenItExists() { Product product = new
      Product("Keyboard", 49.99); product.setId(1L);
      when(productRepository.findByIdReturnsProductWhenItExists(1L)).thenReturn(Optional.of(product
      )); Optional<Product> result = productService.findById(1L);

      assertTrue(result.isPresent());
      assertSame(product, result.orElseThrow());
    assertEquals(1L, result.orElseThrow().getId());
    verify(productRepository).findById(1L); }

    @Test
    void findByIdReturnsEmptyWhenProductDoesNotExist() {
      when(productRepository.findByIdReturnsEmptyWhenProductDoesNotExist(404L)).thenReturn(Optional.empty());

      Optional<Product> result = productService.findById(404L);

      assertTrue(result.isEmpty());
      verify(productRepository).findById(404L);
    }
  }
```

Integration Testing(Added API tests using MySQL Testcontainers for product creation/retrieval, invalid input, and missing products in `ProductIntegrationTest.java`.)

```
    package com.example.ci_pipeline;
import static org.hamcrest.Matchers.is;
    import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
    import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
    import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.jsonPath;
    import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

    import org.junit.jupiter.api.Test;
    import org.springframework.beans.factory.annotation.Autowired;
    import org.springframework.boot.test.context.SpringBootTest;
```

```
import
org.springframework.boot.webmvc.test.autoconfigure.AutoConfigureMockMvc;
import org.springframework.context.annotation.Import; import
org.springframework.http.MediaType;
import org.springframework.test.web.servlet.MockMvc; import
org.springframework.test.web.servlet.MvcResult;

@SpringBootTest(properties = "spring.datasource.password=password")
@AutoConfigureMockMvc
@Import(TestcontainersConfiguration.class)
class ProductIntegrationTest {

    @Autowired private
    MockMvc mockMvc;

    @Test
    void createProductThenRetrieveIt() throws Exception {
        MvcResult creationResult = mockMvc.perform(post("/api/products")
                .contentType(MediaType.APPLICATION_JSON)
                .content("""
                        {"name":"Integration keyboard","price":49.99} """))
            .andExpect(status().isCreated())
            .andExpect(jsonPath("$.id").isNumber())
            .andExpect(jsonPath("$.name", is("Integration keyboard")))
            .andExpect(jsonPath("$.price", is(49.99)))
            .andReturn();

        String id = com.jayway.jsonpath.JsonPath.read(
            creationResult.getResponse().getContentAsString(), "$.id").toString();
        mockMvc.perform(get("/api/products/{id}", id))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.id").value(id))
            .andExpect(jsonPath("$.name", is("Integration keyboard")))
            .andExpect(jsonPath("$.price",  is(49.99)));
    }

    @Test
    void rejectProductWithBlankName() throws Exception {
        mockMvc.perform(post("/api/products")
                .contentType(MediaType.APPLICATION_JSON)
                .content("""
                        {"name":" ","price":10.0}
                        """))
            .andExpect(status().isBadRequest())
            .andExpect(jsonPath("$.title",  is("Invalid  product")));
    }
```

```
    @Test
    void returnNotFoundForUnknownProduct() throws Exception {
        mockMvc.perform(get("/api/products/9223372036854775807"))
                .andExpect(status().isNotFound());
    }
}
```

Pact provider verification(Added a Pact contract and a test that checks the running API against it: `ProductContractVerificationTest.java` and `ProductClient-ProductService.json`.)

```
package com.example.ci_pipeline;

import org.junit.jupiter.api.BeforeEach; import
org.junit.jupiter.api.TestTemplate; import
org.junit.jupiter.api.extension.ExtendWith;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import
org.springframework.boot.test.web.server.LocalServerPort;
import org.springframework.context.annotation.Import; import
org.springframework.jdbc.core.JdbcTemplate;

import au.com.dius.pact.provider.junit5.HttpTestTarget; import
au.com.dius.pact.provider.junit5.PactVerificationContext;
import au.com.dius.pact.provider.junitsupport.Provider; import
au.com.dius.pact.provider.junitsupport.State;
import au.com.dius.pact.provider.junitsupport.loader.PactFolder; import
au.com.dius.pact.provider.spring.junit5.PactVerificationSpringProvider;
import com.example.ci_pipeline.product.Product; import
com.example.ci_pipeline.repository.ProductRepository;

@Provider("ProductService")
@PactFolder("src/test/resources/pacts")
@SpringBootTest( webEnvironment =
    SpringBootTest.WebEnvironment.RANDOM_PORT,
    properties = "spring.datasource.password=password")
@Import(TestcontainersConfiguration.class)
@ExtendWith(PactVerificationSpringProvider.class)
class ProductContractVerificationTest {

    @LocalServerPort
    private int port;

    @Autowired
    private ProductRepository productRepository;

    @Autowired
    private JdbcTemplate jdbcTemplate;
```

```
    @BeforeEach
    void configureTarget(PactVerificationContext context) {
      context.setTarget(new HttpTestTarget("localhost", port));
      product1Exists();
    }

    @TestTemplate
    void verifyPactInteraction(PactVerificationContext context)
    { context.verifyInteraction(); }

    @State("product 1 exists")
    public void product1Exists() { jdbcTemplate.execute("TRUNCATE
      TABLE product");
      productRepository.saveAndFlush(new Product("Keyboard", 49.99));
    }
}
```

Continuous integration workflow added in ci.yml to run Maven verification on pushes and pull requests

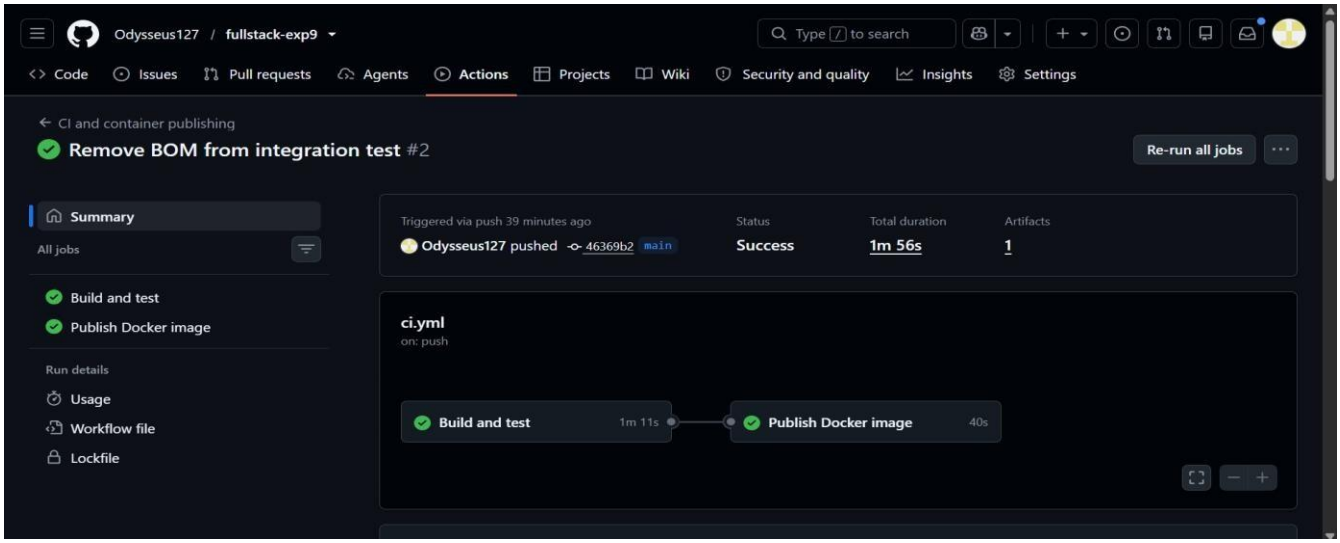
```
name: CI on:
push:
pull_request:

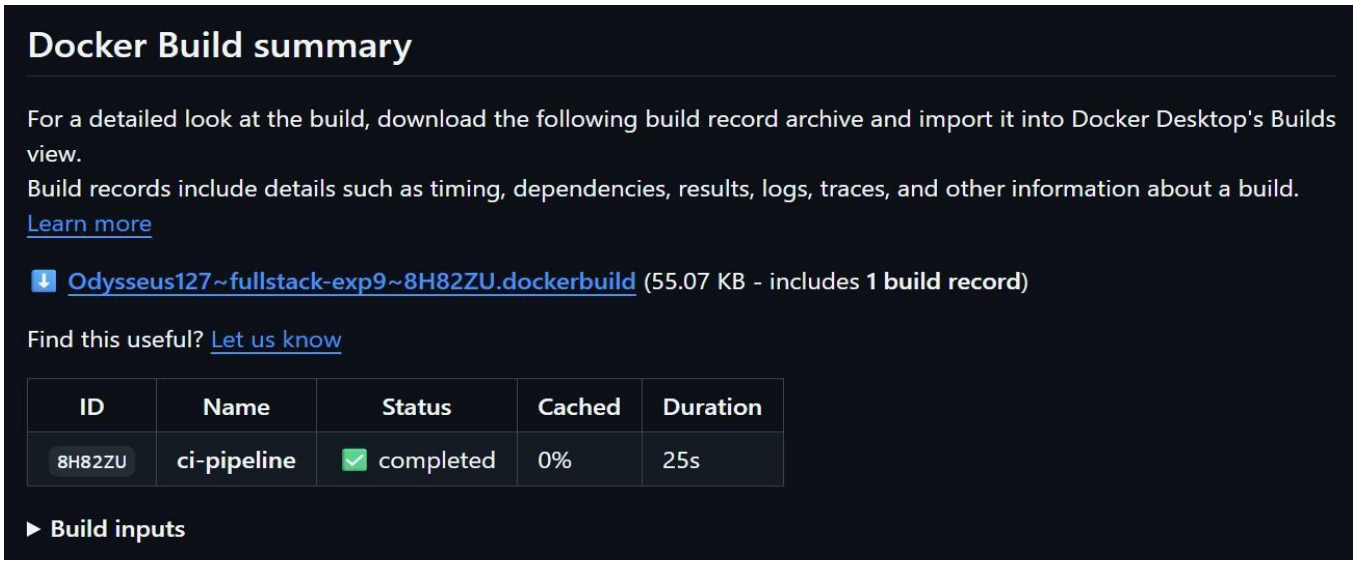
permissions:
  contents: read
jobs:
  build-and-test:
    name: Build and test runs-on: ubuntu-
    latest timeout-minutes: 20 steps:
      - name: Check out source uses:
        actions/checkout@v4

      - name: Set up Java 21 uses:
        actions/setup-java@v4 with:
          distribution: temurin
          java-version: '21' cache:
          maven

      - name: Build and run tests run: bash
        ./mvnw --batch-mode --no-transfer-
        progress verify
```

7) OUTPUT





8) LEARNING OUTCOMES

After completing this experiment, I learned to:

- Explain how CI detects issues early by building and testing code changes automatically.
- Build a Spring Boot application with Maven in a repeatable way.
- Write unit tests for service logic using JUnit and Mockito.
- Write integration tests that exercise API and database behavior with Testcontainers.
- Use Pact to verify that a service satisfies a defined API contract.
- Configure GitHub Actions to run verification on pushes and pull requests.
- Interpret CI results and fix failures such as compilation errors or encoding problems.

CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

Experiment – 9.2

1) **AIM:** To design and implement a Continuous Deployment pipeline with containerization and automated release strategies.

2) OBJECTIVE

1. To understand continuous Deployment concepts.
2. To containerize applications using Docker.
3. To implement canary deployment strategies.
4. To automate deployment using GitHub Actions.

3) SOFTWARE REQUIREMENT

1. Java(JDK 17+)
2. Docker Hub / Container registry
3. IDE(VSCode/Eclipse)
4. Docker
5. GitHub actions

4) THEORY

Continuous Deployment (CD) is the process of automatically deploying applications after successful testing. It ensures that new features and updates are delivered quickly and reliably.

In this experiment, applications are packaged using Docker, which allows them to run consistently across different environments. Docker containers include the application along with all required dependencies, eliminating environment-related issues.
GitHub Actions is used to automate the deployment process. When code is pushed, the pipeline builds the application, runs tests, and deploys it automatically.

A canary deployment strategy is implemented where new changes are released to a small group of users first. If the system performs well, the deployment is expanded to all users, reducing the risk of failures.
This approach ensures faster, safer, and automated software delivery.

5) IMPLEMENTATION/PROCEDURE

1. Create Dockerfile for application
2. Build Docker image
3. Push image to container registry
4. Configure GitHub Actions workflow
5. Automate build and deployment steps
6. Implement canary deployment strategy

6) CODE

Dockerfile builds the app with Java 21 and packages it in a Java 21 runtime image

```
FROM eclipse-temurin:21-jdk AS build
WORKDIR /workspace COPY
. .
RUN chmod +x ./mvnw && ./mvnw --batch-mode --no-transfer-progress -DskipTests package FROM eclipse-temurin:21-jre
WORKDIR /app
COPY --from=build /workspace/target/ci-pipeline-0.0.1-SNAPSHOT.jar app.jar
EXPOSE 8080
```

```
ENTRYPOINT ["java", "-jar", "app.jar"]
```

compose.yaml starts the app and MySQL, waiting for MySQL's health check before starting the app.

```
services: mysql:
```

```
    image:                  mysql:8.4
    environment:
      MYSQL_DATABASE: cipipeline
      MYSQL_USER: ${DB_USERNAME:-cipipeline}
      MYSQL_PASSWORD: ${DB_PASSWORD:-local-dev-password}
    MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD:-local-root-password} volumes:
      -          mysql-
    data:/var/lib/mysql healthcheck:
      test: ["CMD-SHELL", "mysqladmin ping -h localhost -u root -p\"$$MYSQL_ROOT_PASSWORD\""]
      interval: 5s timeout: 5s retries: 20

  app:
    build: .
    ports:
      -          "8080:8080"
    environment: DB_URL:
  jdbc:mysql://mysql:3306/cipipeline?useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC
      DB_USERNAME: ${DB_USERNAME:-cipipeline}
      DB_PASSWORD: ${DB_PASSWORD:-local-dev-password} depends_on:
      mysql:
        condition: service_healthy

volumes:
  mysql-data:
```

ci.yml publishes a commit-SHA and branch-tagged image to GHCR after tests pass on a push.

```
      name: CI and Cloud Run deployment
on: push: pull_request:

      permissions:
        contents: read
jobs:
      build-and-test:
        name: Build and test
        runs-on:      ubuntu-
        latest        timeout-
        minutes: 20

        steps:
          - name: Check out source
            uses: actions/checkout@v4

          - name: Set up Java 21 uses: actions/setup-
            java@v4 with:
              distribution:
              temurin        java-
```

```
          version: '21' cache:
          maven
          cache-dependency-path: ci-pipeline/pom.xml

      - name: Build and run tests working-directory:
        ci-pipeline
        run: bash ./mvnw --batch-mode --no-transfer-progress verify

  publish-image:
    name: Publish image to Artifact Registry if: github.event_name ==
    'push' && github.ref == 'refs/heads/main' needs: build-and-test
    runs-on: ubuntu-latest permissions:
      contents: read
    id-token: write outputs: image: ${{
    steps.image.outputs.image }}

    steps:
      - name: Check out source uses:
        actions/checkout@v4

      - name: Authenticate to Google Cloud
        id: auth uses: google-github-
        actions/auth@v2 with:
          workload_identity_provider: ${{ vars.GCP_WORKLOAD_IDENTITY_PROVIDER
          }} service_account: ${{ vars.GCP_SERVICE_ACCOUNT }} token_format:
          access_token

      - name: Set up Google Cloud CLI
        uses: google-github-actions/setup-
        gcloud@v2

      - name: Configure Docker for Artifact
        Registry run: gcloud auth configure-
        docker "${{ vars.GCP_REGION }}-
        docker.pkg.dev" --quiet

      - name: Define image id: image env:
          PROJECT_ID: ${{ vars.GCP_PROJECT_ID }}
          REGION: ${{ vars.GCP_REGION }}
          REPOSITORY: ${{ vars.GAR_REPOSITORY }}
          COMMIT_SHA: ${{ github.sha }} run: echo "image=${REGION}-
        docker.pkg.dev/${PROJECT_ID}/${REPOSITORY}/ci-
pipeline:${COMMIT_SHA}" >> "$GITHUB_OUTPUT"

      - name: Set up Docker Buildx uses:
        docker/setup-buildx-action@v3
```

```
    - name: Build and publish image uses:
      docker/build-push-action@v6 with:
        context:  ./ci-pipeline  file:
        ./ci-pipeline/Dockerfile
        push: true
        tags: ${{ steps.image.outputs.image }}

deploy-canary:
  name: Canary deploy to Cloud Run if: github.event_name == 'push'
  && github.ref == 'refs/heads/main' needs: publish-image runs-on:
  ubuntu-latest timeout-minutes: 15 concurrency:
    group: cloud-run-${{ vars.CLOUD_RUN_SERVICE }} cancel-in-progress:
    false
  permissions:
    contents: read
  id-token: write env:
    PROJECT_ID: ${{ vars.GCP_PROJECT_ID }}
    REGION: ${{ vars.GCP_REGION }}
    SERVICE: ${{ vars.CLOUD_RUN_SERVICE }}
    RUNTIME_SERVICE_ACCOUNT: ${{ vars.CLOUD_RUN_RUNTIME_SERVICE_ACCOUNT }}
    IMAGE: ${{ needs.publish-image.outputs.image }} CANARY_TAG:
    canary-${{ github.sha }}

  steps:
    - name: Authenticate to Google Cloud
      uses:  google-github-actions/auth@v2
      with:
        workload_identity_provider:  ${{  vars.GCP_WORKLOAD_IDENTITY_PROVIDER  }}
        service_account: ${{ vars.GCP_SERVICE_ACCOUNT }}

    - name: Set up Google Cloud CLI uses:
      google-github-actions/setup-
      gcloud@v2

    - name: Find current serving revision id:
      stable
      shell: bash run:
      |
        set -euo pipefail
        SERVICE_EXISTS="$(gcloud run services list \
      --project "$PROJECT_ID" \
      --region "$REGION" \
      --filter="metadata.name=$SERVICE" \
      --format='value(metadata.name)')"

        if [[ "$SERVICE_EXISTS" == "$SERVICE" ]]; then
      REVISION="$(gcloud run services describe "$SERVICE" \
```

```
            --project "$PROJECT_ID" \
            --region "$REGION" \
          --format='value(status.traffic[0].revisionName)')" if
        [[ -z "$REVISION" ]]; then echo "Cloud Run service exists but has no
            serving revision." >&2 exit 1
        fi
          echo  "revision=$REVISION"  >>  "$GITHUB_OUTPUT"
         else
          echo  "revision="  >>  "$GITHUB_OUTPUT"
         fi


      - name: Deploy canary revision without production traffic
        shell: bash run: |
         set -euo pipefail
         gcloud run deploy "$SERVICE" \
        --project "$PROJECT_ID" \

        --region "$REGION" \
        --platform managed \ --image
        "$IMAGE" \
        --service-account "$RUNTIME_SERVICE_ACCOUNT" \
            --set-secrets                              "DB_URL=product-db-url:latest,DB_USERNAME=product-
dbusername:latest,DB_PASSWORD=product-db-password:latest" \
        --allow-unauthenticated \
        --no-traffic \
        --tag "$CANARY_TAG" \
        --quiet


      -    name:    Smoke-test    canary
        revision id: smoke shell: bash run:
        |
         set -euo pipefail
         CANARY_URL="$(gcloud run services describe "$SERVICE" \
        --project "$PROJECT_ID" \
        --region "$REGION" \
            --format=json | jq -r --arg tag "$CANARY_TAG" \
             '.status.traffic[] | select(.tag == $tag) | .url')"
       if [[ -z "$CANARY_URL" || "$CANARY_URL" == "null" ]]; then echo
        "Could not resolve the Cloud Run canary URL." >&2 exit
            1
         fi
         echo "url=$CANARY_URL" >> "$GITHUB_OUTPUT"
         curl --fail --silent --show-error --retry 6 --retry-connrefused \
        --retry-delay 5 "$CANARY_URL/"
```

```
- name: Shift 10% traffic to canary
 if: steps.stable.outputs.revision !=''
 shell: bash run: |
   set -euo pipefail
   CANARY_REVISION="$(gcloud run services describe "$SERVICE" \
 --project "$PROJECT_ID" \
 --region "$REGION" \
     --format=json | jq -r --arg tag "$CANARY_TAG" \
       '.status.traffic[] | select(.tag == $tag) | .revisionName')" gcloud
   run services update-traffic "$SERVICE" \
 --project "$PROJECT_ID" \
 --region "$REGION" \
 --to-revisions="${{ steps.stable.outputs.revision }}=90,${CANARY_REVISION}=10" \
     --quiet
   sleep 30
   curl --fail --silent --show-error \
 "$(gcloud run services describe "$SERVICE" \
       --project "$PROJECT_ID" \
       --region "$REGION" \
       --format='value(status.url)')/"

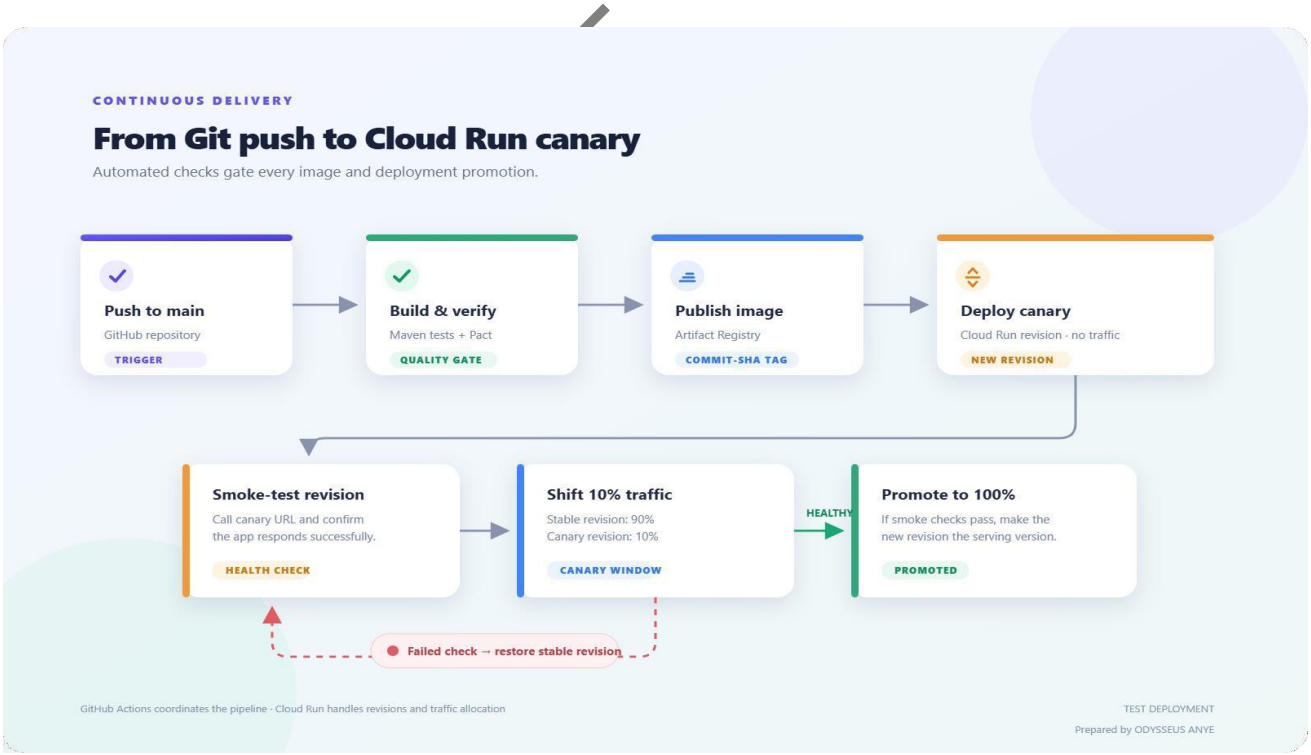
- name: Promote canary to 100% traffic
 shell: bash run: |
   set -euo pipefail
   CANARY_REVISION="$(gcloud run services describe "$SERVICE" \
 --project "$PROJECT_ID" \
 --region "$REGION" \
     --format=json | jq -r --arg tag "$CANARY_TAG" \
       '.status.traffic[] | select(.tag == $tag) | .revisionName')" gcloud
   run services update-traffic "$SERVICE" \
 --project "$PROJECT_ID" \
 --region "$REGION" \
 --to-revisions="${CANARY_REVISION}=100" \
 --quiet

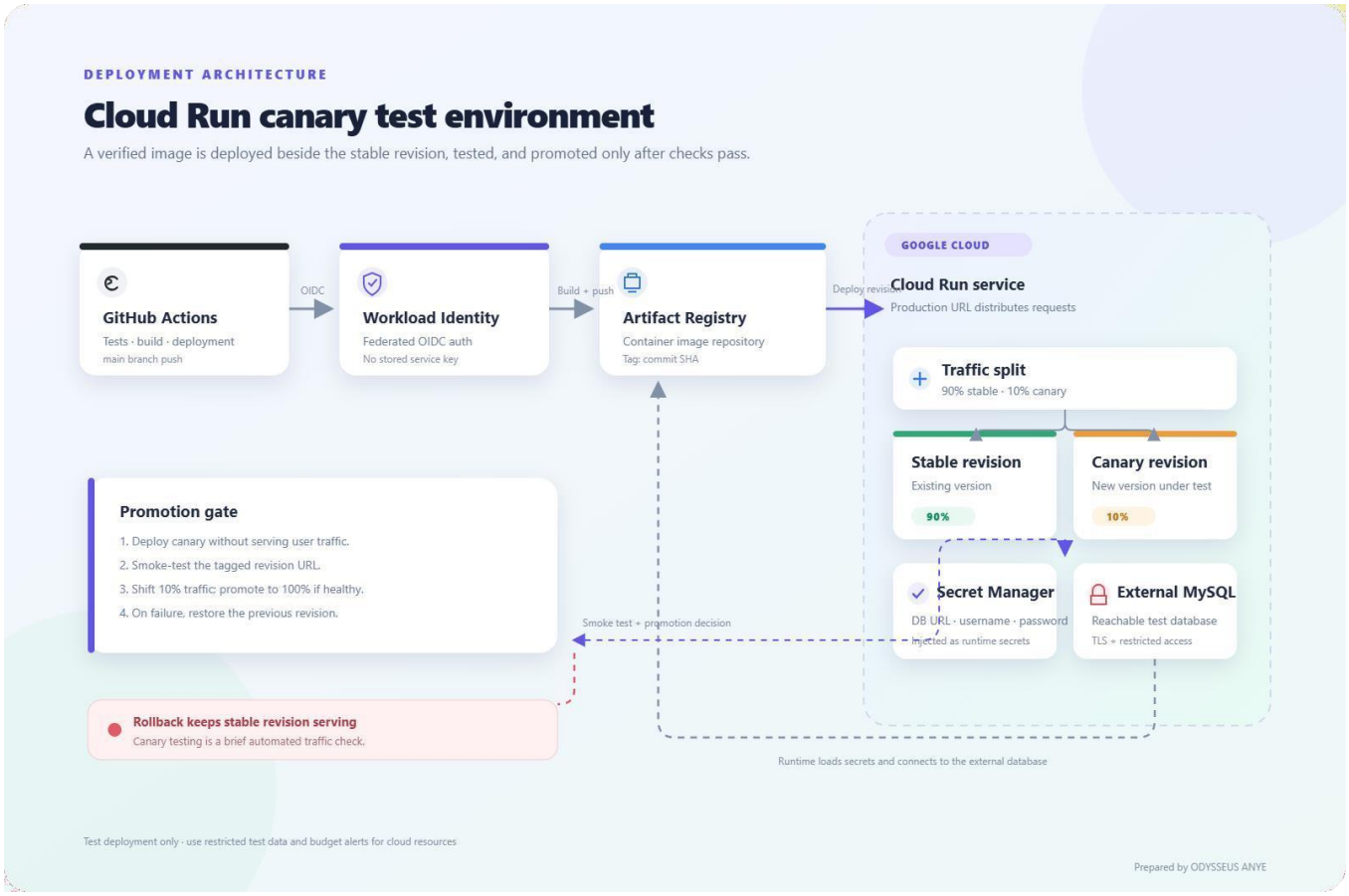
- name: Show deployed service URL
 if:
 sc
 ce
 ss(
 )
 ru
 n: |
   gcloud run services describe "$SERVICE" \
 --project "$PROJECT_ID" \
 --region "$REGION" \
```

```
--format='value(status.url)'

- name: Restore previous revision if deployment fails if: failure() && steps.stable.outputs.revision != '' shell: bash run: |
  gcloud run services update-traffic "$SERVICE" \
--project "$PROJECT_ID" \
--region "$REGION" \
--to-revisions="${{ steps.stable.outputs.revision }}=100" \
--quiet
```

7) OUTPUT





8) LEARNING OUTCOMES

After completing this experiment, I learned to:

- Explain how CD automates packaging and release after CI succeeds.
- Build a Docker image for a Spring Boot application using a multi-stage Dockerfile.
- Run an application and MySQL locally with Docker Compose.
- Publish versioned container images to a registry such as Artifact Registry.
- Authenticate GitHub Actions to Google Cloud using Workload Identity Federation rather than long-lived keys.
- Deploy an application revision to Cloud Run and manage runtime configuration with Secret Manager. Explain and demonstrate canary deployment by gradually shifting traffic to a new revision.